

第 16 章: Function Basics (函数基础)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 第 13 到 15 章我们学的都是"从上到下、一次跑完"的过程式语句; 本章开始进入函数的世界——把一段代码打包成可重复调用的"工具"。函数是 Python 中代码复用 (code reuse) 与程序设计 (program design) 最基本的载体, 也是后续作用域规则 (第 17 章)、参数传递 (第 18 章)、函数式工具 (第 19 章)、生成器与协程 (第 20 章), 乃至第六部分类与面向对象的基石。本章重点: def 是运行时执行的语句、函数是对象 (一等公民)、参数按赋值传递 (引用语义)、多态 (polymorphism)。

16.1 开篇引言 (本章导览)

英文原文

In Part III, we studied basic procedural statements in Python. Here, we'll move on to explore a set of additional statements and expressions that we can use to create functions of our own.

In simple terms, a function is a package of code invoked by name. It labels and groups a set of statements so they can be run more than once in a program. A function also can compute a result value, and lets us s

pecify parameters that serve as inputs and may differ each time the function's code is run. Wrapping an operation in a function makes it a generally useful tool, which we can apply in a variety of contexts.

On a more pragmatic level, functions are the alternative to programming by cutting and pasting—rather than having multiple redundant copies of an operation's code, we can factor it into a single function. In so doing, we reduce our future work radically: if the operation must be changed later, we have only one copy to update in the function, not many scattered throughout the program.

Functions are also the most basic program structure Python provides for maximizing code reuse, and lead us to the larger notions of program design. As you'll see, functions let us split complex systems into manageable parts. By implementing each part as a function, we make it both reusable and easier to code.

Table 16-1 abstractly previews the function-related tools we'll study in this part of the book—a set that includes call expressions, two ways to make functions (`def` and `lambda`), two ways to manage scope visibility (`global` and `nonlocal`), two ways to send results back to callers (`return` and `yield`), and tools to pause for results (`async` and `await`). While this comprises a feature-rich topic, you'll find that its commonly used core is straightforward.

表 16-1. 函数相关的语句和表达式 (Table 16-1. Function-related statements and expressions)

语句或表达式	示例
调用表达式 (Call expressions)	<code>myfunc('hack', tool=python, versions)</code>
<code>def</code>	<code>def printer(message): print('Hello', message)</code>
<code>return</code>	<code>def adder(a, b=1, c): return a + b + c[0]</code>
<code>lambda</code>	<code>funcs = [lambda x: x2, lambda x: x3]</code>
<code>global</code>	<code>x = 'old' def changer(): global x; x = 'new'</code>
<code>nonlocal</code>	<code>def outer(): x = 'old' def changer(): nonlocal x; x = 'new'</code>
<code>yield</code>	<code>def squares(x): for i in range(x): yield i 2</code>
生成器表达式 (Generator expressions)	<code>(i 2 for i in range(x))</code>
<code>async/await</code>	<code>async def consumer(a, b): await producer(b)</code>
装饰器和注解 (Decorators and annotations)	<code>@tracer def func(a:'hack' = None) -> None</code>

中文翻译

在第 3 部分 (Part III) , 我们学习了 Python 中基础的过程式语句。现在, 我们将继续探索一组额外的语句和表达式, 用来创建我们自己的函数。简单来说, 函数 (function) 就是一段按名称调用的代码包。它给一组语句贴上标签并组织起来, 使它们可以在程序中运行多次。函数还能计算一个结果值, 并允许我们指定参数 (parameters) 作为输入——这些参数在函数代码每次运行时都可能不同。把一个操作封装进函数, 它就变成了一个通用的工具, 可以应用在各种不同的场景 (contexts) 中。从更务实的层面讲, 函数是"

复制粘贴式编程"的替代方案——与其让一个操作的代码存在多份冗余副本，不如把它提炼（factor）成单个函数。这样做会极大地减少未来的工作量：如果这个操作以后要修改，我们只需要更新函数里的那一份代码，而不是散落在整个程序中的许多份。函数也是 Python 提供的用于最大化代码复用（code reuse）的最基本程序结构，并把我们将引向更大的程序设计（design）理念。你会看到，函数让我们把复杂的系统拆分成可管理的部分。把每个部分实现为函数，它既可以被复用，也更容易编写。表 16-1 抽象地预览了本书这一部分将要研究的与函数相关的工具——包括调用表达式、两种创建函数的方式（def 和 lambda）、两种管理作用域可见性的方式（global 和 nonlocal）、两种把结果送回调用方的方式（return 和 yield），以及用于暂停等待结果的工具（async 和 await）。虽然这是一个特性丰富的主题，但你会发现它常用的核心部分其实很直白。

深度理解

·核心概念：本章是"函数世界观"的首次亮相。全书后面一切高级话题——生成器、协程、装饰器、类方法——本质上都是"函数"这一概念的延伸。作者在开头就给出一张全景图（表 16-1），就像旅行出发前先看地图。

·底层实现：Python 中的函数不是"源码片段"而是运行时对象。def 执行时会在堆内存（heap）中创建一个函数对象（function object），它打包了函数的代码（字节码，bytecode）和参数元数据。这也是为什么函数可以赋值给别的名字、放进列表、甚至作为参数传递。

·设计原因：Lutz 反复强调"factor into a single function"

n"（提炼为单个函数）——这是软件工程"DRY（Don't Repeat Yourself，不要重复自己）"原则在 Python 里的体现。代码每重复一份，将来修改就要多改一处，bug 概率成倍增长。

·解决什么实际问题：程序越来越大之后，"复制粘贴"无法维护。函数让系统可以分而治之（divide and conquer）：把"做披萨"拆成"揉面、擀皮、加料、烘烤"一个个子任务，每个子任务单独实现、单独调试。

·初学者误区：以为 def 是"写进文件就被编译好"的静态定义。实际上 def 和普通赋值语句一样，执行到它时函数才诞生。另外别被表 16-1 的 async/await、nonlocal 吓到——它们是高级话题，常用的核心（def、return、参数）非常直观。

16.2 为什么要用函数？（Why Use Functions?）

英文原文

Before we get into the details, let's establish a clearer picture of what functions are all about. Functions are a nearly universal program-structuring device. You may have come across them before in other languages, where they may have been called subroutines or procedures. As a brief introduction, functions serve two primary development roles, and serve as the basis of other tools: Maximizing reuse and minimizing redundancy.

ncy As in most programming languages, Python functions are the simplest way to package logic you may wish to use in more than one place and more than one time. Up until now, almost all the code we've been writing has run immediately. Functions allow us to defer and generalize code to be used arbitrarily many times later. Because they allow us to code an operation in a single place and use it in many others, functions are also a factoring tool: they enable us to reduce code redundancy in our programs, and thereby reduce maintenance effort. Dividing and conquering Functions also provide a tool for splitting systems into pieces that have well-defined roles and manageable scopes. For instance, to make a pizza from scratch, you would start by mixing the dough, rolling it out, adding toppings, baking it, and so on. If you were programming a pizza-making robot, functions would help you divide the overall "make pizza" task into smaller parts—one function for each subtask in the process. Because it's easier to implement the smaller tasks in isolation than it is to implement the entire process at once, this makes larger tasks more practical. Implementing object methods In general, functions are about procedure—how to do something, rather than what you're doing it to. Nevertheless, when paired with an implied subject, they can also be used to code object-specific behavior known as methods. You'll see why this distinction matters in Part VI, when we start making n

ew objects with classes. As you'll find, classes are largely just packages of the functions you'll learn to code here.

中文翻译

在深入细节之前，我们先建立一个关于函数更清晰的整体图景。函数是一种近乎通用的程序组织装置。你可能在别的语言中见过它们，那些语言里它们可能被称为子程序（subroutines）或过程（procedures）。作为简短介绍：函数扮演两个主要开发角色，同时也是其他工具的基础：最大化复用、最小化冗余和大多数编程语言一样，Python 函数是封装逻辑的最简单方式——当你希望某段逻辑在多个地方、多次使用时。到目前为止，我们写过的几乎所有代码都是立即执行的。函数让我们把代码延后（defer）并泛化（generalize），以便日后任意多次使用。因为函数让我们把操作写在单一位置、在多处使用，它也是一种提炼（factoring）工具：它帮助我们减少程序中的代码冗余，从而减少维护成本。分而治之函数还提供了把系统拆分成多个部分的工具，每个部分有明确定义的角色和可控的边界。比如，要完全手工做一张披萨，你会先和面、擀皮、加料、烘烤，等等。如果你在给一个“做披萨机器人”编程，函数会帮你把整个“做披萨”任务拆成更小的部分——流程中每个子任务一个函数。因为隔离实现小的子任务比一次性实现整个流程更容易，这让更大的任务变得可行。实现对象的方法一般来说，函数关乎的是过程（procedure）——怎么做某件事，而不是对什么做。不过，当它与一个隐含的主体配对时，也可以用来编写对象特有的行为，称为方法（methods）。在第 6 部分（Part VI），当我们开始用类（classes）创建新对

象时，你会明白这个区别为什么重要。你会发现，类在很大程度上就是这里你要学会的函数的打包组合。

深度理解

- 核心概念：函数的两大角色——复用（reuse）与分解（divide and conquer）。前者解决"代码重复"问题，后者解决"系统太大"问题；第三个角色"方法"是函数在面向对象里的延伸。
- 底层实现："把代码延后（defer）"在底层是什么意思？def 语句运行时，解释器把函数体编译成字节码并保存在函数对象内部；这些字节码不会立即执行，只有调用（call）发生时才会被压入调用栈（call stack）执行。这就是"延迟执行"的机制本质。
- 设计原因："分而治之"源于人类认知局限：一次只能理解有限复杂度的东西。函数把复杂度封装进名字里，调用方只关心"做什么"（接口），不关心"怎么做"（实现）——这是抽象（abstraction）的雏形，也是后面类的哲学基础。
- 解决什么实际问题：维护噩梦。一个业务逻辑复制粘贴了 5 处，某天需求变了，漏改一处就是线上 bug。函数让"改一处，处处生效"。披萨机器人例子说明：任务分解后每个子任务可单独测试，总复杂度急剧下降。
- 初学者误区：把"方法（method）"和"函数（function）"混为一谈。现在只需记住：方法=和对象绑定的函数；细节在第六部分展开。另外别以为函数只和"代码复用"有关——它更是系统设计的基本单元。

16.3 函数编码概述 (Function Coding Overview)

英文原文

Although it wasn't made very formal, we've already used functions in earlier chapters. For instance, we called the built-in `open` function to make a file object, invoked the `len` built-in function to ask for the number of items in a collection object, and employed tools like `zip` and `range` for iteration tasks. In this chapter, we will begin exploring how to write new functions in Python. Functions we write behave the same way as the built-ins we've already seen: they are called in expressions, are passed values, and return results. But writing new functions requires the application of a few additional tools and ideas that haven't yet been introduced. Moreover, functions behave very differently in Python than they do in compiled languages like C.

中文翻译

虽然我们没有正式定义它，但在前面的章节里我们已经用过函数了。比如，我们调用内建 (built-in) 的 `open` 函数来创建文件对象，调用内建的 `len` 函数来询问集合对象中元素的数量，还使用过 `zip` 和 `range` 这类工具完成迭代任务。本章我们将开始探索如何编写 Python 中新的函数。我们写的函数和已经见过的内建函数行为完全一致：在表达式中被

调用、被传入值、返回结果。但编写新函数需要运用一些尚未介绍的工具和思想。此外，函数在 Python 中的行为和它在 C 这类编译语言中非常不同。

深度理解

·核心概念：函数不是新东西——你从第 1 章开始就天天在用（print、len、open、range 都是函数）。本章只是把"使用者"身份切换为"创造者"身份。学习捷径：先把自己写过的调用代码逆向理解一遍。

·底层实现：内建函数（built-in functions）和用户自定义函数在 Python 内部都是函数对象，前者由 C 实现（解释器启动时预装），后者由 Python 字节码实现（def 运行时创建）。调用机制完全统一——这也是"面向接口"设计的一个实例。

·设计原因：Python 刻意让"自己写的函数"和"内建函数"行为一致，降低学习成本：你已经会调用 open，就会调用自己写的函数。

·解决什么实际问题：课程到这一步，前面的代码都是"线性脚本"。真实软件需要把逻辑封装、参数化、可复用，函数就是跨越"脚本"到"程序"门槛的必经之路。

·初学者误区：误以为函数是 C 语言里那种"必须全部定义好才能编译"的东西。记住作者的预告：Python 函数的行为和编译语言（如 C）差异巨大——差异的核心就是"def 是运行时语句"，16.9 节会专门展开。

16.4 基本函数工具 (Basic Function Tools)

英文原文

As a road map and brief rundown on what we'll study in this part of the book, here are the basic components and concepts in Python's function toolbox: `def` creates a function and assigns it to a name. Python functions are coded with `def` statements. When Python reaches and runs a `def`, it generates a new function object and assigns it to the function's name. As with all assignments, the function name becomes a reference to the function object. There's nothing magical about the name of a function—as you'll see, the function object can be assigned to other names, stored in a list, and so on. Function objects may also have arbitrary user-defined attributes attached to them to record data. `def` is executable code. Unlike functions in compiled languages, `def` is an executable statement—your function does not exist until Python runs its `def`. In fact, it's legal (and even occasionally useful) to nest `def` statements inside `if` statements, for loops, and even other `defs`. In typical usage, `def` statements are coded at the top level of module files, and are automatically run to make functions when their modules are imported. `return` sends a result object back to the caller. When a function is called, the caller normally stops until the function finishes its work and returns control to the

he caller. Functions that compute a value send it back to the caller with a return statement, and the returned value becomes the result of the function call. A return without a value simply returns to the caller (and sends back None, the default result). lambda creates a function but returns it as a result. Function objects may also be created with the lambda expression, a feature that allows us to code inline function definitions in places where a def statement won't work syntactically. Like def, functions made this way are created when the lambda is reached and run. Because lambda is not typically coded at the top level of a module, though, such functions are made during program runs, not imports. lambda is an optional convenience best seen as a sidebar to the def statement.

中文翻译

作为路线图和本书这一部分将要学习内容的简要概括，下面是 Python 函数工具箱中的基本组件和概念：def 创建函数并把函数赋值给一个名字。Python 函数用 def 语句编写。当 Python 执行到并运行一个 def 时，它会生成一个新的函数对象，并把它赋值给函数名。和所有赋值一样，函数名成为对函数对象的引用。函数的名字没有任何神奇之处——你会看到，函数对象可以被赋给其他名字、存进列表，等等。函数对象上还可以挂载任意的用户自定义属性（attributes），用于记录数据。def 是可执行代码。与编译语言中的函数不同，def 是一条可执行语句——在 Python 运行你的 def 之前，你的函数并不存在。事实上，把 def 语句嵌

套进 if 语句、for 循环，甚至其他 def 里，都是合法的（偶尔还很有用）。典型用法中，def 语句写在模块文件的顶层，当模块被导入时它们会被自动运行以创建函数。return 把结果对象送回调用方。当一个函数被调用时，调用方通常会停下来，直到函数完成工作并把控制权交还给调用方。计算出一个值的函数用 return 语句把它送回调用方，返回值成为函数调用的结果。不带值的 return 只是返回调用方（并送回 None，即默认结果）。lambda 创建函数但把函数作为结果返回。函数对象也可以用 lambda 表达式创建，这个特性让我们可以在 def 语句语法上无法工作的地方编写内联（inline）函数定义。和 def 一样，这种方式创建的函数也是在执行到并运行 lambda 时才创建。不过，由于 lambda 通常不会写在模块顶层，这类函数是在程序运行期间创建的，而不是导入期间。lambda 是一个可选的便利设施，最好把它看作 def 语句的旁注。

深度理解

- 核心概念：函数的四大基本件——def（创建）、return（送回结果）、lambda（内联创建）、函数名只是引用。其中"def 是可执行语句"是全书最重要的概念之一。
- 底层实现：def 执行时，解释器构造一个 PyFunctionObject：记录函数代码字节码、参数元数据、所在模块的命名空间引用（globals）、默认值等。函数名字符串仅仅通过模块/局部的名字字典（namespace dict）指向这个对象。属性（attribute）为什么能挂上去？因为函数对象内部有 dict，本质上是"对象即字典"。
- 设计原因：把 def 设计成"运行时语句"而不是"编译期定

义”，Python 才可能实现：运行时按条件选择函数版本、函数工厂（返回函数的函数）、动态创建方法等高级技巧。C 语言做不到，这正是 Python 灵活性的来源之一。

·解决什么实际问题：没有“预编译”阶段，意味着 Python 可以做到“走到哪一步算哪一步”——if test 决定定义哪个版本的 func，完全动态。

·初学者误区：a) 以为函数名是“固定的、不能变的名字”——其实它只是字典里的一个键，随便重新赋值；b) 以为 lambda 是“没有名字的 def”——更准确地说它是“没有赋值功能的 def”；c) 忽略 return 时函数返回 None，很多人因此把 None 当结果用而出 bug。

16.5 高级函数工具 (Advanced Function Tools)

英文原文

In addition, Python functions can leverage more advanced tools like the following, which we'll take up after we've introduced the basics: global declares module-level variables that are to be assigned. By default, all names assigned in a function are local to that function and exist only while the function runs. To assign a name in the enclosing module, functions need to list it in a global statement. More generally, names are always looked up in scopes—places where variables are stored—and names are bound to scopes per the lo

cation of assignments in your code. `nonlocal` declares enclosing-function variables that are to be assigned. In the same category as `global`, the `nonlocal` statement allows a function to assign a name that exists in the scope of a syntactically enclosing `def` statement. This allows enclosing functions to serve as a place to retain state—information remembered between function calls—without using shared global names. `yield` sends a result object back to the caller, but remembers where it left off. Functions known as generators may also use the `yield` statement to send back a value and suspend their state such that they may be resumed later, to produce a series of results over time. Along with their generator expression kin, such functions save space and avoid delays just like the built-in iterables we met in the prior part of this book. `await/async` pause a waiting function so that other tasks may run. Functions known as coroutines can suspend their execution until a required result is available. This is an advanced applications-level topic, but allows code to use language tools to get other work done while waiting on a blocking event like slow IO.

中文翻译

此外，Python 函数还可以利用下面这些更高级的工具，我们将在介绍完基础之后再来讨论它们：`global` 声明要赋值的模块级变量。默认情况下，函数内赋值的所有名字都局限于该函数，并且只在函数运行期间存在。要在外层模块中赋

值某个名字，函数需要把它列在 `global` 语句里。更一般地说，名字总是在作用域（scopes）——变量存储的位置——中被查找，而名字按照代码中赋值语句的位置绑定到相应作用域。`nonlocal` 声明要赋值的（语法上）外层函数的变量。与 `global` 同类，`nonlocal` 语句允许函数给存在于语法上包围它的 `def` 语句作用域中的名字赋值。这让外层函数可以充当保存状态（state）——两次函数调用之间记住的信息——的场所，而无需使用共享的全局名字。`yield` 把结果对象送回调用方，但记住它离开的位置。被称为生成器（generators）的函数还可以用 `yield` 语句送回一个值并挂起自己的状态，以便之后恢复，从而随时间产生一系列结果。和它们的近亲生成器表达式（generator expression）一样，这类函数节省空间、避免延迟，和本书前面部分见过的内建可迭代对象如出一辙。`await/async` 暂停一个等待中的函数，让其他任务得以运行。被称为协程（coroutines）的函数可以挂起执行，直到所需的结果可用。这是一个高级的应用程序级话题，但它允许代码在等待慢速 IO 这类阻塞事件时，利用语言工具去干其他工作。

深度理解

·核心概念：四个高级工具各解决一个问题——`global/nonlocal` 控制“名字去哪赋值”（作用域），`yield` 让函数“暂停-恢复”，`await` 让函数“让路”。本章只做预告，详情分别在后续章节展开。

·底层实现：`yield` 之所以能“记住离开的位置”，是因为生成器函数被调用时返回的生成器对象保存了执行帧（frame，含局部变量、指令指针）——暂停时冻结帧，恢复时

从断点继续。协程 (async/await) 在此基础上引入事件循环 (event loop) 调度。

·设计原因：作用域设计成"默认局部" (local by default) 是为了安全：函数内部随手写个临时变量不会意外污染全局空间——就像每个人在自己工位上干活，不碰别人的抽屉。

·解决什么实际问题：yield 让"按需生产结果"成为可能（读 1TB 文件不必全装入内存）；await 让"等待网络响应时不干等"成为可能（单线程也能并发）。

·初学者误区：a) 以为函数里能随便改全局变量——默认不行，必须 global 声明，否则你只是在"新建局部变量"；b) 把生成器和普通函数混用——普通函数 return 一次就结束，生成器 yield 多次。这些是第 17、20 章的重点，现在有个印象即可。

16.6 通用函数概念 (General Function Concepts)

英文原文

Finally, along the way we'll explore a handful of core concepts that span functions of all kinds: Arguments are passed by assignment (object reference). In Python, arguments are passed to functions by assignment—which, as we've learned, means by object reference. As you'll see, in Python's model the caller and function share objects by references, but there is no name

aliasing. Changing an argument name within a function does not also change the corresponding name in the caller, but changing passed-in mutable objects in place can change objects shared by the caller, and serve as a function result. Arguments are passed by position, unless you say otherwise. Values you pass in a function call match argument names in a function's definition from left to right by default. For flexibility, function calls can also pass arguments by name with `name=value` keyword syntax and unpack arbitrarily many arguments to send with `args` and `args` starred-value notation. Function definitions use the same syntax forms to specify argument defaults and collect arbitrarily many arguments received. Arguments, return values, and variables are not declared. As with everything in Python, there are no type constraints on functions. In fact, nothing about a function needs to be declared ahead of time: you can pass in arguments of any type, return any kind of object, and so on. As a consequence, a single function can often be applied to a variety of object types—any objects that have a compatible interface (support for expected methods and expressions) will do, regardless of their specific types. This makes code flexible by design. Attributes, annotations, and decorators support advanced roles. Functions can optionally be augmented with both user-defined attributes and other tools that extend their roles. Annotations of arguments and results, for example,

can serve a variety of goals. Among these, type hinting (noted in Chapter 6) gives suggested object types but is unused by Python itself and simply serves as a weighty form of documentation, which we won't cover further in this book. More usefully, functions can also be prefixed with @ decorators that add extra layers of logic but are used in ways that merit largely deferring until Part VI when we've also learned about classes. If some of the preceding words didn't sink in, don't worry—we'll explore most of these concepts with real code in this part of the book. Let's get started by expanding on some of the basics by coding a few abstract examples.

中文翻译

最后，这一路上我们还会探索几个贯穿所有种类函数的核心概念：参数按赋值（对象引用）传递。在 Python 中，参数是通过赋值传给函数的——正如我们学过的，这意味着按对象引用（object reference）传递。你会看到，在 Python 的模型中，调用方和函数通过引用共享对象，但不存在名字别名（aliasing）问题。在函数内修改一个参数的名字，并不会同时修改调用方中对应的名字；但对传入的可变对象进行就地修改，却可以改变调用方共享的对象，并充当函数的结果。除非另有说明，参数按位置传递。默认情况下，函数调用中传入的值从左到右匹配函数定义中的参数名。为了灵活，函数调用还可以用 name=value 关键字语法按名传递参数，并用 args 和 args 星号值记法解包任意数量的参数发送。函数定义则使用相同的语法形式来指定默认值并收

集接收到的任意数量的参数。参数、返回值、变量都不需要声明。和 Python 中一切事物一样，函数没有任何类型约束。事实上，函数的任何方面都无需提前声明：你可以传入任意类型的参数、返回任何种类的对象，等等。因此，同一个函数常常可以应用于多种对象类型——任何拥有兼容接口（interface，即对预期方法和表达式的支持）的对象都可以，无论它们的具体类型是什么。这让代码天生就灵活。属性、注解和装饰器支持高级角色。函数可以可选地通过用户自定义属性和其他工具得到增强，扩展其角色。例如，参数和结果的注解（annotations）可以服务于多种目标。其中，类型提示（type hinting，第 6 章提到过）给出建议的对象类型，但 Python 本身并不使用它，它只是作为一种厚重的文档形式，本书不再展开。更有用的是，函数还可以加上 @ 装饰器（decorators）前缀，为函数添加额外的逻辑层，但它的用法值得推迟到第 6 部分——那时我们也学过类了。如果前面有些词没有立刻领会，别担心——本书这一部分会用真实的代码探索这些概念中的大多数。现在就开始吧，我们先写一些抽象的例子来展开基础内容。

深度理解

·核心概念：四条通则定义了"Python 函数哲学"：按引用传参（无值拷贝）、位置参数优先、零声明（动态类型）、可被增强（属性/注解/装饰器）。其中第一条"参数按赋值传递"是最容易被初学者理解错的概念。

·底层实现："按对象引用传递" = 函数调用时，实参对象的内存地址被赋值给形参名。形参只是"贴在新名字上的标签"。关键区分：给形参重新赋值（重新贴标签）不影响调用

方；但顺着引用修改对象内容（如 `list.append`）会影响调用方，因为两边贴的是同一个对象的标签。这和 C 的"按值传递/按指针传递"都不同——Python 是"按对象引用传递（传对象、不传变量）"。

·设计原因：零声明 + 多态 = 无需为每种类型写一个重载版本。C++/Java 里 `times(2, 4)` 和 `times("Py", 4)` 是不同函数（重载/泛型）；Python 一个函数通吃，靠的是对象自己实现运算符。

·解决什么实际问题：面向接口（interface）编程使代码"插拔兼容"——只要对象支持函数期望的操作（协议，protocol），就能直接用，无论它是内置类型还是以后别人写的类。这把函数的适用范围从"作者预期的类型"扩展到"整个对象宇宙"。

·初学者误区：a) 以为"不声明类型" = "不检查错误"——其实运行时会在操作不兼容时报 `TypeError`；b) 以为函数内修改参数（可变对象）和修改参数名是一回事——前者影响调用方，后者不影响；c) 把类型注解当成强制类型检查——它只是文档，解释器完全不强制。

16.7 def 语句 (def Statements)

英文原文

The `def` statement creates a function object and assigns it to a name. Its general format and usage is as follows: `def name(arg1, arg2,... argN):` # Define a function statements # Function body `name(val1, val2,... valN`

) # Call it later in an expression As with all compound statements, `def` consists of a header line followed by a block of other statements, usually indented (though a simple statement after the colon works as usual). The statement block becomes the function's body—that is, the code Python executes each time the function is later called. The `def` header line specifies a function name that is assigned the new function object, along with a list of zero or more arguments (sometimes called parameters) enclosed in parentheses (even for zero arguments). The argument names in the header are assigned to the objects passed in parentheses at the point of call elsewhere in your code. To call the function later in your program and run its body, you can use the function name assigned by `def`, passing in zero or more values (objects) to match the arguments listed in the `def` header. A call expression can also denote the function with other types of object references; it need not necessarily be the name coded in the `def` header.

中文翻译

`def` 语句创建一个函数对象 (function object) 并把它赋值给一个名字。它的一般格式和用法如下：
`python def name(arg1, arg2,... argN):` # 定义一个函数
`statements` # 函数体
`name(val1, val2,... valN)` # 稍后在表达式中调用它
与所有复合语句一样，`def` 由一行头部 (header) 和随后的语句块组成，语句块通常缩进 (冒号后跟一条简单语句也和

平常一样可行)。这个语句块成为函数的函数体 (body)——也就是每次函数被调用时 Python 要执行的代码。def 的头部行指定一个函数名，新函数对象被赋给这个名字，同时还有一个零个或多个参数 (arguments，有时称为形参 parameters) 的列表，放在圆括号中 (即使参数为零也要有括号)。头部中的参数名会被赋值给程序别处调用点传入括号中的那些对象。要在程序中稍后调用函数并运行它的函数体，你可以使用 def 赋出的函数名，传入零个或多个值 (对象) 去匹配 def 头部列出的参数。调用表达式也可以使用其他类型的对象引用来表示这个函数；不一定是 def 头部里写下的那个名字。

代码分析

```
def name(arg1, arg2, ... argN):           #
    statements                             #

name(val1, val2, ... valN)                #
```

·解释器如何处理：当解释器逐行执行到 def 行时：1) 先读取函数名 name 与参数列表，编译函数体语句为字节码；2) 在堆上创建函数对象 (内含字节码、name、defaults 等元数据)；3) 在当前命名空间字典中执行 name = 函数对象的赋值。注意这一步不执行函数体。直到代码遇到 name(val1, ...) 调用行，解释器才创建调用帧 (frame)：把实参对象按顺序赋给形参名，然后执行函数体字节码。

·设计思想："头部+缩进块"的语法与 if/for/while 完全一致，学习成本为零；而"零参数也必须有括号" (def f():) 是为了调用时语法统一——f 是引用函数对象，f() 才是执

行。

·易错点：函数名只是个引用。def f(): ... 之后，g = f 合法，f 也可被重新指向其他对象——“调用”只看你手上持有哪个函数对象。

深度理解

·核心概念：def 的完整语义 = “创建对象 + 赋值名字”两步。头部 (header) 定义接口 (名字和形参)，函数体 (body) 定义实现 (每次调用执行的代码)。

·底层实现：CPython 中函数对象对应 C 结构体 PyObject，持有：funccode (编译后的代码对象，含字节码与常量表)、funcglobals (模块全局字典引用)、funcdefaults (默认值元组)。调用时 PyEvalEvalFrame 创建新帧执行 funccode，帧中局部命名空间是独立字典。

·设计原因：为什么头部要写参数名？因为调用时“实参对象赋值给形参名”需要名字作为接收端——这正是“按赋值传参”语法的落点，也是第 18 章所有参数匹配规则 (位置、关键字、默认值、args) 的载体。

·解决什么实际问题：函数把“接口”与“实现”分离：调用方只需知道 name(val1, ...) 怎么写，不必关心函数体怎么实现；函数作者可以自由修改实现细节而不惊动调用方。

·初学者误区：a) 忘记空参数也必须写括号 def f():; b) 在 def 前面加类型声明 (int def f()) ——Python 没有这种语法，类型由对象决定；c) 认为 def 一行一写就“注册”了函数——不，必须执行到该行才创建。

16.8 return 语句 (return Statements)

英文原文

Function bodies often contain one or more return statements that make sense only inside a def: `def name(arg1, arg2,... argN):...return result` # The result of the call expression

The Python return statement can show up anywhere in a function body. When reached, it ends the function call and sends a result back to the caller to serve as the result of the call expression. The return statement consists of an optional object-value expression that gives the result. If the value is omitted, return sends back a None by default. The return statement itself is optional too; if it's not present, the function exits when the control flow falls off the end of the function body. Technically, a function without a return statement also returns the None object automatically, but this return value is usually ignored at the call, by using a call-expression statement of Chapter 11. Functions may also contain yield statements used to produce a series of values over time, as well as await and async for/with statements used to suspend the function's execution, but we'll defer discussion of these more advanced tools until we survey generator and coroutine topics in Chapter 20.

中文翻译

函数体经常包含一个或多个 `return` 语句，它们只在 `def` 内部才有意义：`python def name(arg1, arg2, ... argN): ... return result` # 调用表达式的结果 Python 的 `return` 语句可以出现在函数体的任何位置。一旦执行到它，它就结束这次函数调用，并把结果送回调用方，作为调用表达式的结果。`return` 语句由一个可选的对象值表达式组成，这个表达式给出结果。如果省略值，`return` 默认送回 `None`。`return` 语句本身也是可选的；如果没有它，当控制流走出函数体末尾时函数就退出了。严格来说，没有 `return` 语句的函数也会自动返回 `None` 对象，但这个返回值通常在调用处被忽略——通过使用第 11 章的调用表达式语句（`call-expression statement`）来调用。函数还可以包含用于随时间产生一系列值的 `yield` 语句，以及用于挂起函数执行的 `await` 和 `async for/with` 语句，但我们把这些更高级工具的讨论推迟到第 20 章，那时我们会系统地介绍生成器和协程话题。

代码分析

```
def name(arg1, arg2, ... argN):  
    ...  
    return result                #
```

·解释器如何处理：执行 `return result` 时，解释器先求值 `result` 表达式得到对象，然后：1) 把该对象写入当前调用帧的返回值槽位；2) 销毁帧（释放局部变量名）；3) 控制权交还调用方，调用表达式整体"变为"这个对象。若函数体执行完仍无 `return`，解释器在帧结束时自动压入 `None`

作为返回值——这就是"跌出函数体末尾"时的默认返回。

·设计思想：return 在函数体任何位置都有效，意味着它天然是一种"提前退出"控制流——配合 if 可以实现"遇到边界条件直接返回"，避免深嵌套 else。这是函数式写法 (guard clause) 的基础。

·易错点：return 与 return None 等价；return 与 yield 不能混用（函数只要含 yield 就变成生成器，普通 return 变成 StopIteration 的载体）。

深度理解

·核心概念：return 是函数与外界沟通结果的唯一正式通道（另一条是 16.5 预告的 yield）。"调用表达式的结果"——即 `x = times(3.14, 4)` 中 `x` 拿到的值。

·底层实现：返回值在字节码层是 RETURNVALUE 指令：它把栈顶对象弹给调用者，然后弹出当前执行帧。帧被弹出意味着全部局部变量立即失效——这直接解释了 16.15 节"res 这个名字会消失，但结果对象活着"。

·设计原因：为什么"没写 return 就自动返回 None"？因为 Python 的每条表达式语句都必须有值，函数调用本身也是表达式；统一默认值让语法简单（函数不必每个出口都写 return）。这就是 C 语言 void 概念在 Python 中的等价物。

·解决什么实际问题：无 return 函数（如只负责打印、写文件的过程）与有 return 函数（纯计算）可以共存，编程模型统一：print 函数返回 None，但没人因此崩溃——因为调用者通常忽略它。

·初学者误区：a) 忘记写 `return` 导致函数"神秘地"返回 `None`（最常见 bug 之一：if 分支里返回了值，else 分支忘了）；b) 以为 `return` 后面的代码不执行是"错误"——那是正常的设计手段（提前返回）；c) 把 `return` 写在函数体外——语法错误，`return` 只能出现在 `def` 内。

16.9 `def` 在运行时执行 (`def` Executes at Runtime)

英文原文

The Python `def` is a true executable statement: when it runs, it creates a new function object and assigns it to a name. (Remember, all we have in standard Python is runtime; there is no such thing as a separate compile time.) Because it's a statement, a `def` can appear anywhere a statement can—even nested in other statements. For instance, although `defs` are normally top-level code run when the module enclosing them is imported, it's also completely legal to nest a function `def` inside an `if` statement to select between alternative definitions:

```
if test: def func(): ... # Define func this way
else: def func(): ... # Or else this way ... func() # Call the version
selected and built
```

One way to understand this code is to realize that the `def` is much like an `=` statement: it simply assigns a

name (like `func` here) at runtime. Unlike in compiled languages such as C, Python functions do not need to be fully defined before the program runs. More generally, `defs` are not run until they are reached, and the code inside `defs` is not run until the functions are later called. Because function definition happens at runtime, there's nothing special about the function name. What's important is the object to which it refers. For example:

```
othername = func # Assign function object  
othername() # Call func again
```

Here, the function was assigned to a different name and called through the new name. Like everything else in Python, functions are just objects; they are recorded explicitly in memory at program execution time. In fact, besides calls, functions allow arbitrary attributes to be attached to record information for later use:

```
def func(): ... # Create + assign function object  
func() # Call object via its name  
func.attr = value # Attach attributes
```

In other words, functions are first-class objects, to borrow a term introduced in Chapter 9. While they don't support some operations that other objects do, functions inhabit the same category as every object. As you'll see, passing them about and storing them in other objects is both syntactically legal and surprisingly useful.

中文翻译

Python 的 `def` 是一条真正的可执行语句：它运行时，会创建一个新的函数对象并把它赋给一个名字。（记住，标准 Python 中我们只有运行时（runtime）；不存在单独的编译期（compile time）这种东西。）因为它是语句，`def` 可以出现在任何语句能出现的地方——甚至嵌套在其他语句里。比如，虽然 `def` 通常是顶层代码、在其所在模块被导入时运行，但把函数 `def` 嵌套进 `if` 语句来选择不同的定义版本，也完全合法：`python if test: def func(): ... # 这样定义 func else: def func(): ... # 或者这样定义 func ... func()` # 调用被选中并构建出来的那个版本理解这段代码的一种方式 是认识到：`def` 很像一条 `=` 赋值语句——它只是在运行时给一个名字（比如这里的 `func`）赋值。与 C 这类编译语言不同，Python 函数不需要在程序运行前被完整定义。更一般地说，`def` 直到被执行到才会运行，而 `def` 内部的代码直到函数之后被调用才会运行。因为函数定义发生在运行时，函数名没有任何特殊之处。重要的是它引用的那个对象。例如：`python othername = func # 赋值函数对象 othername()` # 再次调用 `func` 这里，函数被赋值给了另一个名字，并通过新名字被调用。和 Python 中的其他一切事物一样，函数只是对象；它们在程序执行时被明确地记录在内存中。事实上，除了调用之外，函数还允许挂上任意属性（attributes）来记录信息供以后使用：`python def func(): .. # 创建并赋值函数对象 func() # 通过名字调用对象 func.attr = value # 挂上属性` 换句话说，借用第 9 章引入的术语，函数是一等对象（first-class objects）。虽然函数不支持某些其他对象支持的操作，但函数与每一个对象处于同一

类别。你会看到，把函数传来传去、把它们存进其他对象里，不仅在语法上合法，而且出奇地有用。

代码分析

```
if test:
    def func(): ...      # func
else:
    def func(): ...      # func
...
func()                  #
```

·解释器如何处理：这不是“编译期声明两个函数，运行时选一个”——Python 根本没有编译期声明。执行到 if 分支时，只运行被选中分支里的那条 def：解释器当场编译该函数体、创建函数对象、执行 func = <新函数对象> 赋值，覆盖任何旧值。另一个分支的 def 从未被运行，其函数体甚至不会被打包成对象。这正应了“def 像 = 语句”：条件成立才赋值。

·设计思想：这种写法就是运行时多态定义（也叫“可选实现”）：比如按操作系统选择不同的平台函数实现，或按配置选择算法。C 语言做不到，因为 C 在链接期就定死了符号。

·易错点：如果 if/else 两个分支都没选中（如 test 抛异常或值使两个条件都为假——此处没有 else 时），后续 func() 会抛 NameError。使用前务必确保定义路径必然执行。

深度理解

- 核心概念：本章最核心的顿悟点：`def` = 运行时赋值。"函数不需要预先定义"意味着 Python 把"定义"变成了一种普通操作，函数对象可以随意被改名、被引用、被挂属性。
- 底层实现：`func.attr = value` 为什么可行？函数对象内部有一个 `dict` 字典（属性字典），`func.attr = value` 在字节码层是 `STOREATTR` 指令，本质是往该字典写键值。同理 `othername = func` 只是往当前命名空间字典里加一个新键。没有"符号表固定"的约束，一切皆字典操作。
- 设计原因：一等对象（first-class object）设计使函数可以像整数、字符串一样被传递和组合——这是函数式编程（`map/filter/sort key`、装饰器、回调）在 Python 里能实现的前提。C 语言函数只能通过函数指针勉强模拟，Java 早期则完全要包装类。
- 解决什么实际问题：回调（callback）机制：把"做什么"作为函数参数传给框架（如排序的 `key`、GUI 的事件处理）；策略模式：用字典把字符串映射到函数，代替一长串 `if/elif`；装饰器：给函数包一层新逻辑。
- 初学者误区：a) 以为"定义顺序无关"——错误！`def` 只有被执行到才创建函数，调用必须发生在定义之后（除非把函数调用放在函数体内部，延迟到运行时）；b) 以为 `func.attr = value` 是 Python 新特性或魔法——它只是普通对象属性，C 函数（内建函数）部分不支持；c) 把"名字"和"对象"混为一谈：改名、重新赋值都不影响函数对象本

身。

16.10 lambda 创建匿名函数 (lambda Makes Anonymous Functions)

英文原文

In addition to `def`, you can make a new function with the `lambda` expression. It's coded and might be used like this: `name = lambda arg1, arg2,... argN: expression name(val1, val2,... valN)` Like `def`, `lambda` makes a new function object to be called later. It begins with the word `lambda`, followed by an arguments-list header that works the same as in `def` but is coded without parentheses. Unlike `def`, the code after the `:` in `lambda` is a single expression—it cannot contain statements and is the implied body and return value of the function that `lambda` makes. We don't need to say `return` in a `lambda` (and we can't). Also unlike `def`, `lambda` does not itself assign the new function to a name, but simply returns it as the result of the whole `lambda` expression. This snippet manually assigns the function to a name through which it is called, but that's optional: the result might also be saved in another object or passed for use elsewhere. Because of this, `lambda` is usually called an "anonymous" function—one that's unnamed. You'll learn more about `lambda` later and see it in action along the way. But your first question

may be, Why have a version of `def` whose code is limited to just one expression? In short, `lambda` can be used in places that `def` cannot, including in calls and object literals where we want to embed small bits of deferred and runnable code. While `def` handles larger tasks and also supports embedding functions by name, `lambda` is an optional amenity in simpler roles. And if you're keeping track, we've now seen four expression equivalents for more general statements—the `lambda` for `def`, the `:=` named assignment for `=`, the `if/else` ternary for `if`, and the comprehension for `for`. Although `lambdas` are limited to expressions, they can use any of these expression equivalents to code assignments, logic, and loops. All these expression forms were accumulated over time, but are convenient alternatives in practical code.

中文翻译

除了 `def`，你还可以用 `lambda` 表达式创建新函数。它的写法和用法如下：`python name = lambda arg1, arg2, ... argN: expression` `name(val1, val2, ... valN)` 和 `def` 一样，`lambda` 创建一个稍后会被调用的新函数对象。它以关键字 `lambda` 开头，后面跟着一个参数列表头部（header），作用与 `def` 中的相同，但书写时不带圆括号。与 `def` 不同，`lambda` 中冒号：后面的代码是一个单一的表达式——它不能包含语句，并且它就是 `lambda` 所创建函数的隐含函数体和返回值。我们不需要在 `lambda` 中说 `return`（也不能说）。同样与 `def` 不同，`lambda` 本身不会把新函数赋给

一个名字，而只是把函数作为整个 lambda 表达式的结果返回。上面这段代码手动把函数赋给一个名字并通过它调用，但这只是可选的：结果也可以存进另一个对象，或传给别处使用。正因如此，lambda 通常被称为“匿名”（anonymous）函数——没有名字的函数。关于 lambda，你以后会学到更多，并会在路上看到它的实际应用。但你的第一个问题可能是：为什么需要这个代码仅限于一个表达式的 def 变体？简而言之，lambda 可以用在 def 语法上无法工作的地方，包括调用参数和对象字面量里——那些我们想内嵌一小段“延后执行的、可运行的代码”的位置。def 处理更大的任务，也支持按名字内嵌函数，而 lambda 在更简单的角色里是一个可选的便利品。如果你一直在留意，我们已经见到了更通用语句的四个表达式等价物——lambda 对应 def，:= 命名赋值表达式对应 =，if/else 三元表达式对应 if，推导式（comprehension）对应 for。虽然 lambda 仅限于表达式，但它可以使用所有这些表达式等价物来编码赋值、逻辑和循环。这些表达式形式是随时间逐渐累积起来的，但在实际代码中是方便的替代品。

代码分析

```
name = lambda arg1, arg2, ... argN: expression
name(val1, val2, ... valN)
```

·解释器如何处理：执行 lambda 行时，解释器创建函数对象并返回它（与 def 的“赋值”不同，这里只有“返回”这一步）；示例中的 name = ... 是外层赋值把返回值接住。函数对象内部，expression 被编译为函数体的唯一表达式，函数调用时求值该表达式、其结果自动成为返回值（字节码层面等价于 def name(...): return expression）。

·设计思想：lambda 是"表达式版 def"：表达式 (expression) 可以出现在任何值能出现的地方 (列表里、函数参数里、字典值里)，而语句 (statement) 不能。这就是 lambda 存在的全部理由——语法位置限制的破解。

·易错点：lambda 体内不能有赋值 (如 lambda x: x += 1 非法)、不能有 if 语句 (但可以用三元 if/else 表达式)、不能有多条语句。可读性差时，优先用普通 def——作者也明说 lambda 是"可选便利品"。

深度理解

·核心概念：lambda = "没有名字 + 没有赋值副作用 + 只有表达式"的 def。它把"定义函数"压缩成一条表达式，可以嵌在任意表达式语境中。

·底层实现：lambda 生成的函数对象与 def 生成的几乎完全一样 (同样的 PyFunctionObject、同样的调用机制)，唯一显著差别是 name 固定为 "<lambda>" 且没有 funccode.cofilename 之外的额外源码定位。所以它并不是"更轻量的魔法"，只是语法糖。

·设计原因：为什么限制为单表达式？因为多语句表达式在语法上根本无法定义 (语句没有值)。Python 的选择是"宁可限制能力，也不破坏表达式语法的纯净"。作为补偿，语言陆续加了三元表达式、推导式、海象运算符 (:=) 三个"表达式等价物"，凑齐四件套。

·解决什么实际问题：最典型场景是给排序/过滤类高阶函数传行为：sorted(items, key=lambda x: x['age'])、list (map(lambda x: x2, nums))——函数被当作"一段延后执

行的小逻辑"直接塞进调用里，不必为此单独写 def 再命名。

·初学者误区：a) 在 lambda 里尝试写语句导致 SyntaxError；b) 认为 lambda 性能更好——它和 def 没有性能差异；c) 在循环里用 lambda 捕获循环变量 ([lambda: i for i in range(3)] 得到三个返回 2 的函数) ——这是闭包延迟绑定的经典坑，第 17 章作用域会解释；d) 为了"用 lambda"而用 lambda，牺牲可读性——默认优先 def。

16.11 第一个示例：定义与调用 (A First Example: Definitions and Calls)

英文原文

Apart from their runtime flavor (which tends to seem most novel to programmers with backgrounds in compiled languages), Python functions are straightforward to use. Let's code a first real example to demonstrate the fundamentals, from both sides of the function equation: definition (the def or lambda that creates a function) and calls (the expressions that tell Python to run the function's body). Definition Here's a definition typed interactively that defines a function named times, which returns the product of its two arguments (it's not much, but it demos the basic bits). As usual "..." prompts are omitted here for copy and paste:

```
>>> def times(x, y): # Create and assign function ret
urn x y # Body executed when called When Python re
aches and runs this def, it creates a new function obj
ect that packages the function's code, and assigns thi
s object to the name times. Typically, such a stateme
nt is coded in a module file and runs when the enclos
ing file is imported; for something this small, though,
the interactive REPL suffices. As an aside, a lambda ca
n have the same effect if we assign its result to a nam
e, though it's typically used in more focused roles tha
n this, and often not as top-level code. Try the def, la
mbda, or both if you're working along—they both m
ake function objects that work the same:
```

```
>>> times = lambda x, y: x y Calls Both def and lam
bda make a function but do not call it. After either ha
s run, you can call (i.e., run) the function in your prog
ram by adding parentheses after the function's name.
The parentheses may optionally contain one or more
object arguments, to be passed (i.e., assigned) to the
names in the function's header:
```

```
>>> times(2, 4) # Arguments in parentheses 8
```

This expression passes two arguments to `times`. As mentioned previously, arguments are passed by assignment, so in this case the name `x` in the function header is assigned the object `2`, `y` is assigned `4`, and the function's body is run. For this function, the body is just a `def`'s return statement or a `lambda`'s expression,

that sends back the result as the value of the call expression. The returned object was printed here interactively (as in most languages, 2×4 is 8 in Python), but if we needed to use it later we could instead assign it to a variable. For example:

```
>>> x = times(3.14, 4) # Save the result object
>>> x
12.56
```

Now, watch what happens when the function is called a third time, with very different kinds of objects passed in:

```
>>> times('Py', 4) # Functions are "typeless"
'PyPyPyPy'
```

This time, our function means something wholly different. In this third call, a string and an integer are passed to `x` and `y`, instead of two numbers. Recall that `times` works on both numbers and sequences; because we don't constrain the types of variables, arguments, or return values in Python, we can use `times` to either multiply numbers or repeat sequences. In other words, what our `times` function means and does depends on what we pass into it. This is a core idea in Python (and perhaps the key to using the language well), which merits a separate callout here.

中文翻译

除了运行时风味 (runtime flavor, 对编译语言背景的程序员来说它往往最新奇) 之外, Python 函数用起来很直白。

让我们编写第一个真实示例来演示基本原理，从函数等式（equation）的两侧来看：定义（definition，创建函数的 `def` 或 `lambda`）和调用（calls，告诉 Python 运行函数体的表达式）。定义（Definition）下面是一个交互式输入的、名为 `times` 的函数定义，它返回两个参数的乘积（内容不多，但演示了基本要素）。和往常一样，这里省略了“...”提示符以便复制粘贴：

```
python >>> def times(x, y): # 创建并赋值函数
return x * y # 被调用时执行的函数体
当 Python 执行到并运行这条 def 时，它创建一个打包了函数代码的新函数对象，并把这个对象赋给名字 times。通常这类语句写在模块文件中，在包含它的文件被导入时运行；不过对于这么小的函数，交互式 REPL 就足够了。顺带一提，如果把 lambda 的结果赋给一个名字，它也能达到同样效果——不过 lambda 通常用于比这更聚焦的角色，而且往往不作为顶层代码。如果你在跟着练习，把 def、lambda 或两者都试一遍——它们创建的函数对象工作起来完全一样：
```

```
python >>> times = lambda x, y: x * y
```

调用（Calls）`def` 和 `lambda` 都会创建函数，但都不会调用它。任一种运行之后，你都可以在函数名后面加上圆括号来调用（也就是运行）程序中的这个函数。圆括号里可以视情况包含一个或多个对象实参，它们会被传递（也就是赋值）给函数头部中的名字：

```
python >>> times(2, 4) # 圆括号里的参数
```

这个表达式把两个参数传给 `times`。如前所述，参数通过赋值传递，所以这里函数头部的名字 `x` 被赋值为对象 `2`，`y` 被赋值为 `4`，然后运行函数体。对这个函数来说，函数体就是 `def` 的一条 `return` 语句，或 `lambda` 的一个表达式，它把结果作为调用表达式的值送回。返回的对象在这里被交互式打印出来（和大多数语言一样，`2 * 4` 在 Python 中是 `8`），但如果我

们之后还要用它，可以把它赋给一个变量。例如：python

```
>>> x = times(3.14, 4) # 保存结果对象>>> x 12.56
```

现在，看看当函数第三次被调用、传入非常不同类型的对象时会发生什么：python

```
>>> times('Py', 4) # 函数是"无类型"的'PyPyPyPy'这一次，我们的函数意味着完全不同的事情。在第三次调用中，传给 x 和 y 的是一个字符串和一个整数，而不是两个数字。回想一下，运算符对数字和序列都有效；因为在 Python 中我们不对变量、参数或返回值的类型设限，所以可以用 times 来乘数字，也可以重复序列。换句话说，我们的 times 函数意味着什么、做什么，取决于我们传什么进去。这是 Python 的一个核心思想（也许正是用好这门语言的关键），值得在这里单独提出来讲。
```

代码分析

```
>>> def times(x, y): #
    return x * y    #

>>> times(2, 4)     #
8

>>> x = times(3.14, 4) #
>>> x
12.56

>>> times('Py', 4)   # ""
'PyPyPyPy'

>>> times('not', 'quite') #
TypeError: can't multiply sequence by non-int of type 'str'
```

·解释器如何处理：调用 times(2, 4) 时，解释器把整数对象 2 和 4 的引用分别赋给形参 x、y，然后进入函数体执行 RETURNVALUE 前的 x y 求值：执行 BINARYOP 指令，该指令查找整数类型的 mul 方法做乘法，栈顶得到对

象 8, return 交还。第三次调用 `times('Py', 4)` 时, 的调度对象变成字符串——字符串的 `mul` 实现是"重复", 于是得到 `'PyPyPyPy'`。 `times('not','quite')` 时字符串的 `mul` 发现重复次数不是整数, 抛出 `TypeError`——错误由对象自己发现并报告, 函数代码根本不需要检查类型。

·设计思想: 是一个"分派机制" (dispatch mechanism)
: 它把控制权路由给正在被处理的对象。同一段函数代码 = 不同的行为, 这正是多态的精髓, 也让函数代码量大幅缩减。

·易错点: `x = times(3.14, 4)` 之后交互式输入 `x` 显示 12.56——别把"保存结果"和"再次调用"混淆: `x` 是结果对象, 不是函数。

深度理解

·核心概念: 函数等式两端的对称性——定义 (`def`/`lambda`) 只负责"制造工具", 调用 (`call`) 才负责"使用工具"。一个函数可以定义一次、调用无数次; 每次调用都重新运行函数体。

·底层实现: REPL 里的 `def` 与模块里的 `def` 没有本质区别——REPL 本身就是"一行一行执行语句的模块"。 `times` 名字存在于交互环境的 `main` 命名空间字典中。 `lambda` 版本创建的函数对象与 `def` 版本唯一可观察的差异是名字元数据, 调用机制完全相同。

·设计原因: "无类型" (typeless) 函数 = 函数不预先声明参数类型, 类型由传入的对象自报家门。这让"同一函数服务于多种类型"成为默认行为而非特例, 直接催生了 16.12

节的多态。

·解决什么实际问题：一个 `times` 同时是"乘法器"和"重复器"。真实场景：一个 `sum` 函数既能加数字也能拼接列表；一个 `render` 函数能渲染任何实现了渲染接口的对象——代码量减半、适用范围加倍。

·初学者误区：a) 从编译语言转来的人总想先声明 `x: int`——Python 不需要也不需要；b) 以为 `times('Py', 4)` 的结果是 `'PyPyPyPy'` 很奇怪——回顾第 4 章：字符串整数就是重复；c) 在函数里写一堆 `type()` 检查"防错"——作者明确指出这会限制函数可用性，错误交给解释器自动检测即可。

16.12 Python 中的多态 (Polymorphism in Python)

英文原文

As we just saw, the very meaning of the expression `x y` in our simple `times` function depends completely upon the kinds of objects that `x` and `y` are—thus, the same function can perform multiplication in one instance and repetition in another. Python leaves it up to the objects to do something reasonable for the syntax. Really, it is just a dispatch mechanism that routes control to the objects being processed. This sort of type-dependent behavior is known as polymorphism, a term we first met in Chapter 4 that essentially means th

at the meaning of an operation depends on the objects being operated upon. Because it's a dynamically typed language, polymorphism runs rampant in Python. In fact, every operation is a polymorphic operation in Python: printing, indexing, the operator, and much more. This is deliberate, and it accounts for much of the language's conciseness and flexibility. A single function, for instance, can generally be applied to a whole category of object types automatically. As long as those objects support the expected interface (a.k.a. protocol), the function can process them. That is, if the objects passed into a function have the expected methods and expression operators, they are plug-and-play compatible with the function's logic. Even in our simple times function, this means that any two objects that support a will work, no matter what they may be, and no matter when they are coded. This function will work on two numbers (performing multiplication), or a string and a number (performing repetition), or any other combination of objects supporting the expected interface—even class-based objects we have not even imagined yet. Moreover, if the objects passed in do not support this expected interface, Python will detect the error when the expression is run and raise an exception automatically. It's therefore usually pointless to code error checking in such code ourselves. In fact, doing so would limit our function's utility, as it would be restricted to work only on objects whos

e types we test for: >>> times('not','quite') TypeError: can't multiply sequence by non-int of type 'str' This turns out to be a crucial philosophical difference between Python and statically typed languages like C++ and Java: in Python, your code is not supposed to care about specific data types. If it does, it will be limited to working on just the types you anticipated when you wrote it, and it will not support other compatible object types coded now or in the future. Although it is possible to test for types with tools like the `type` built-in function, doing so breaks your code's flexibility. By and large, we code to object interfaces in Python, not data types. Of course, some programs have unique requirements, and this polymorphic model of programming means we have to test our code to detect some errors that statically typed compiled languages might be able to detect earlier. In exchange for an initial bit of extra testing, though, we radically reduce the amount of code we have to write, and radically increase our code's flexibility. As you'll come to find with time, it's a resounding net win in practice.

中文翻译

正如我们刚刚看到的，简单 `times` 函数中表达式 `x y` 的含义完全取决于 `x` 和 `y` 是什么种类的对象——因此同一个函数在一种场合执行乘法，在另一种场合执行重复。Python 把“为语法做合理的事”留给了对象自己。真的，只是一个分派机制（dispatch mechanism），把控制权路由到正在被

处理的对象上。这种依赖类型的行为被称为多态 (polymorphism)，我们在第 4 章首次遇到这个术语，它的基本含义是：一个操作的含义取决于被操作的对象。因为 Python 是动态类型语言，多态在 Python 中无处不在。事实上，每一个操作在 Python 中都是多态操作：打印、索引、运算符，以及更多。这是有意为之的，它成就了这门语言的大量简洁性和灵活性。比如，单个函数通常可以自动应用于一整类对象类型。只要这些对象支持预期的接口 (interface，也叫协议 protocol)，函数就能处理它们。也就是说，如果传入函数的对象拥有预期的方法和表达式运算符，它们就能与函数的逻辑即插即用兼容。即使在我们简单的 times 函数里，这也意味着：任何两个支持的对象都能工作，无论它们是什么，无论它们是什么时候写的。这个函数可以用于两个数字 (执行乘法)、或一个字符串和一个数字 (执行重复)、或任何其他支持预期接口的对象组合——甚至我们还没想象到的、基于类的对象。此外，如果传入的对象不支持这个预期接口，Python 会在表达式运行时检测到错误并自动抛出异常。因此，在这种代码里自己编写错误检查通常毫无意义。事实上，这样做反而会限制函数的有用性——函数将只能处理我们显式测试过类型的那些对象：python >>> times('not','quite') TypeError: can't multiply sequence by non-int of type 'str'这其实是 Python 与 C++、Java 这类静态类型语言之间的一个关键哲学差异：在 Python 中，你的代码不应该关心具体的数据类型。如果它关心了，它就会局限于处理你写代码时预期的那些类型，无法支持现在或将来编写出来的其他兼容对象类型。虽然可以用 type 内建函数之类的工具测试类型，但这样做会破坏代码的灵活性。总的来说，在 Python 中我们针对对象的接口编

程，而不是数据类型。当然，有些程序有特殊需求，而且这种多态编程模型意味着我们必须测试自己的代码，以发现静态类型编译语言也许能更早发现的某些错误。不过，用最初多花的一点测试时间，换来了代码编写量的大幅减少和代码灵活性的巨幅提升。随着时间的推移你会发现，在实践中这是不折不扣的大赢。

代码分析

```
>>> times('not', 'quite')
TypeError: can't multiply sequence by non-int of type 'str'
```

·解释器如何处理：调用时 $x='not'$ 、 $y='quite'$ ，函数体执行 $x\ y$ ——`BINARYOP` 指令把操作分派给字符串类型的 `mul`。`str` 的 `mul` 要求乘数是 `int`（重复次数），发现 `'quite'` 不是整数，于是在 C 层设置异常并回卷调用栈，抛出 `TypeError`。注意：错误发生在运行到这一行时，而不是调用前——动态语言“晚绑定（late binding）”的典型表现。函数体里没有任何类型检查代码，错误完全由对象方（`str`）自行声明“我不支持这个接口”。

·设计思想：鸭子类型（duck typing）的雏形——“像鸭子一样叫、像鸭子一样走，就是鸭子”。函数不查类型，只发操作请求；对象必须自己负责响应。这使代码专注业务逻辑，去掉海量类型分支。

·易错点：错误直到运行期才出现，静态类型语言能在编译期抓到的错误，Python 要在测试阶段抓到——所以 Python 项目必须配足测试（这也是作者“换测试保灵活”观点的由来）。

深度理解

·核心概念：多态 = 操作的含义由操作对象决定。每一个操作都是多态的——`print` 能打印一切对象是因为每个对象实现了自己的字符串表示；索引 `[]` 对不同对象意味着取元素/取键。函数因此自动"适配"整类对象，无需重载（overload）。

·底层实现：运算符在字节码层是 `BINARYOP` 指令，运行时通过对象的类型对象（type object）查找对应槽位方法（slot，如 `mul`）来分派——这就是作者说的"dispatch mechanism"。这个查找发生在每次操作执行时，所以任何对象（包括未来才写的类）只要实现了相应方法就能参与运算。

·设计原因：为什么刻意选择"不管类型"？因为接口（interface）比类型（type）更稳定：全世界的对象实现"乘"接口，方式可以千差万别（数字乘、字符串重复、矩阵乘法）。面向接口编程让代码对"未来"开放——写代码时还不存在的对象类型也能被兼容。

·解决什么实际问题：通用工具函数一次编写、处处适用：`len()` 对字符串/列表/字典/文件都有效；自定义类只要实现 `len` 就能被 `len()` 接受；我们的 `intersect` 函数（16.13 节）能交任何可迭代序列。没有多态，这些都要为每种类型重写。

·初学者误区：a) 从 Java/C++ 转来的人习惯"类型先行走天下"，到处写 `isinstance()` 检查——在 Python 里这是反模式；b) 把多态误解为"同一个函数名可以定义多份"（

那是重载，Python 同名只能覆盖)；c) 觉得"不检查类型 = 不严谨"——严谨性由充分的测试保证，而不是由静态检查保证。作者明说：这是 Python 与 C++/Java 的"关键哲学差异" (crucial philosophical difference) 。

16.13 第二个示例：求序列交集 (A Second Example: Intersecting Sequences)

英文原文

Next, let's explore a second function example that does something a bit more useful than multiplying arguments and further illustrates function basics. In Chapter 13, we coded a for loop that collected items held in common in two strings (and called it nearly intersection, except for duplicates). We noted there that the code wasn't as useful as it could be because it was set up to work only on specific variables and could not be rerun later. Of course, we could copy the code and paste it into each place where it needs to be run, but this solution is neither good nor general—we'd still have to edit each copy to support different sequence names, and changing the algorithm would then require changing multiple copies. Definition By now, you can probably guess that the solution to this dilemma is to package the for loop inside a function. Doing so offers a number of advantages: - Putting the code in a function makes it a tool that you can run as many ti

mes as you like. - Because callers can pass in arbitrary arguments, functions are general enough to work on any two sequences (or other iterables) you wish to intersect. - When the logic is packaged in a function, you have to change code in only one place if you ever need to change the way the intersection works. - Coding the function in a module file means it can be imported and reused by any program run on your machine. In effect, wrapping the code in a function makes it a general utility, as captured by Example 16-1. Example 16-1. `inter1.py`

```
def intersect(seq1, seq2): res = [] # Start empty for x
in seq1: # Scan seq1 if x in seq2: # Common item? re
s.append(x) # Add to end
```

`return res` The transformation from the simple code of Chapter 13 to this function is straightforward; we've just nested the original logic under a `def` header and made the objects on which it operates passed-in parameter names. Because this function computes a result, we've also added a `return` statement to send a result object back to the caller. Calls Before you can call a function, you have to make it. To do this, run its `def` statement, either by typing it interactively or by coding it in a module file and importing the file. You can paste it at a REPL as a whole, but we'll import it from a file here (per Chapter 3, make sure you can see it in your REPL by working in the file's folder if needed). Once you've run the `def` by such means, you can call

the function by passing any two sequence objects in parentheses:

```
>>> from inter1 import intersect # Get function from module
>>> s1 = 'HACK'
>>> s2 = 'CHOK'
>>> intersect(s1, s2) # Pass two strings ['H','C','K']
```

Here, we've passed in two strings, and we get back a list containing the characters in common. The algorithm the function uses is simple: "For every item in the first argument, if that item is also in the second argument, append the item to the result." It's a little shorter to say that in Python than in English, but it works out the same. To be fair, our intersect function could probably be quicker (it executes naive nested loops), isn't really mathematical intersection (there may be duplicates in the result), and isn't required at all (as we've seen, Python's set objects provide a built-in intersection operation kicked off with `&`). Indeed, the function could be replaced with a single list comprehension expression, as it exhibits the classic collector-loop code pattern:

```
>>> [x for x in s1 if x in s2] ['H','C','K']
```

Which also works as a loop inside a lambda body expression—despite being five lines in the `def`. This suffices as a demo, but doesn't make much sense in a simple role like this (again, we'll put lambdas to better use later):

```
>>> intersect = lambda seq1, seq2: [x for x in seq1 if x in seq2] >>> intersect(s1, s2) ['H','C','K']
```

However it's coded, though, this function does the job: its single piece of code can apply to an entire range of object types, as the next section explains. In fact, we'll improve and extend this to support arbitrarily many operands in Chapter 18, after we uncover more about argument passing modes.

中文翻译

接下来，让我们探索第二个函数示例——它做的事情比乘两个参数更有用一些，并进一步说明函数基础。在第 13 章，我们写过一个 for 循环，收集两个字符串中共有的条目（我们称之为“近似的交集”，因为不去重）。我们当时指出，这段代码并不像它本可以的那样有用，因为它是为特定变量写死的，以后无法重新运行。当然，我们可以复制代码、粘贴到每个需要运行的地方，但这种方案既不好也不通用——我们仍然需要编辑每一份副本来支持不同的序列名，而且将来要修改算法就得修改多份副本。定义（Definition）到现在，你大概已经猜到解决这个困境的办法是把 for 循环封装进一个函数。这样做有一系列优点：- 把代码放进函数，它就变成了一个可以任意多次运行的工具。- 因为调用方可以传入任意实参，函数足够通用，能对你想交集的任何两个序列（或其他可迭代对象 iterables）工作。- 当逻辑封装在函数中后，如果以后需要改变交集的工作方式，你只需在一处修改代码。- 把函数写进模块文件，意味着它可以被导入，被你机器上运行的任何程序复用。实际上，把代码包进函数，就使它成为一个通用工具，如示例 16-1 所示。示例

16-1. inter1.py python def intersect(seq1, seq2): res = [] # 初始为空 for x in seq1: # 扫描 seq1 if x in seq2: # 共同条目? res.append(x) # 追加到末尾 return res 从第 13 章的简单代码到这个函数的转换是直截了当的；我们只是把原来的逻辑嵌套进一个 def 头部之下，并把操作所依赖的对象变成了传入的参数名。因为这个函数要计算结果，我们还加了一条 return 语句，把结果对象送回调用方。调用 (Calls) 调用函数之前，必须先创建它。为此，运行它的 def 语句——要么交互式输入，要么写进模块文件并导入该文件。你可以把整段代码粘贴到 REPL，但我们这里选择从文件导入（按第 3 章的说法，必要时请确保在文件所在文件夹中工作，以便 REPL 能看到它）。一旦通过这种方式运行了 def，你就可以在圆括号中传入任意两个序列对象来调用函数：python >>> from inter1 import intersect # 从模块获取函数 >>> s1 = 'HACK' >>> s2 = 'CHOK' >>> intersect(s1, s2) # 传入两个字符串 ['H','C','K'] 这里我们传入了两个字符串，得到包含共同字符的列表。函数使用的算法很简单："对第一个参数中的每一个条目，如果它也在第二个参数中，就把它追加到结果里。"用 Python 说比用英语说稍微短一点，但意思完全一样。公平地说，我们的 intersect 函数可能可以更快（它执行的是朴素的嵌套循环）、并不是真正的数学交集（结果中可能有重复）、而且并非必需（正如我们见过的，Python 的集合 set 对象提供了用 & 触发的内建交集运算）。事实上，这个函数可以用一个列表推导式 (list comprehension) 表达式替代，因为它体现了经典的"收集循环" (collector-loop) 代码模式：python >>> [x for x in s1 if x in s2] ['H','C','K'] 它同样可以作为 lambda 函数体里的循环来工作——尽管在 def 里

是五行代码。这足以作为演示，但在这样简单的角色里意义不大（再说一次，我们以后会让 lambda 发挥更好的用途）：
python >>> intersect = lambda seq1, seq2: [x for x in seq1 if x in seq2]
>>> intersect(s1, s2) ['H','C','K']
不过，无论怎么编写，这个函数都完成了任务：它这唯一的一段代码可以应用于整整一个范围的对象类型，下一节会解释。事实上，在第 18 章我们了解了更多参数传递模式之后，会把它改进并扩展为支持任意多个操作数。

代码分析

```
# inter1.py
def intersect(seq1, seq2):
    res = []
    for x in seq1:
        if x in seq2:
            res.append(x)
    return res
```

·解释器如何处理：模块被导入时（from inter1 import intersect），Python 从上到下执行模块代码：执行 def 创建函数对象并绑定到模块命名空间的名字 intersect；然后 from ... import 把该名字的引用复制进交互环境的命名空间。调用 intersect(s1, s2) 时：形参 seq1、seq2 分别被赋值为'HACK'、'CHOK' 对象；函数体中的 for 循环隐式创建迭代器逐字符扫描 seq1（'H'、'A'、'C'、'K'）；每个字符执行 in seq2 成员测试（本质是调 str.contains）；res.append(x) 就地修改列表对象；最后 return res 把列表对象送回调用方。注意 res 是函数局部变量，调用结束后这个名字就消失，但返回的列表对象继续存活在调用方的变量中。

·设计思想：这是"收集循环" (collector-loop) 模式的经典样板——初始化空容器 → 循环筛选 → 返回容器。理解这个模式后，一眼就能认出它能被列表推导式替换：[x for x in seq1 if x in seq2]。三行变一行，语义不变。

·易错点：if x in seq2 对字符串是子串/字符成员测试，对列表是元素测试，对字典是键测试——不同对象的 in 语义不同（这就是多态）。所以这个函数"依赖接口"：第一个参数支持 for 迭代，第二个参数支持 in 测试。

深度理解

·核心概念：把第 13 章的"一次性脚本"升级为"通用工具"的四步操作：取循环代码 → 包进 def → 数据变参数 → 加 return。这是"脚本 → 库"演进的教科书示范。

·底层实现：for x in seq1 在字节码层使用迭代协议：调用 iter(seq1) 得到迭代器对象，循环体反复执行 next() 直到抛 StopIteration。x in seq2 走成员测试协议：优先 contains，否则退化为迭代逐个比较（走 iter/getitem）。第一个参数只需支持迭代，第二个只需支持成员测试——这就是"接口"的具体含义。

·设计原因：为什么费劲把代码包成函数而不是继续复制粘贴？四个优势作者列得很清楚：可重复运行、参数通用化、单点修改、模块级可导入复用。复制粘贴的隐性成本（多副本同步修改）往往在需求变更时才爆发。

·解决什么实际问题：任何需要"两个序列求公共元素"的场景都能用它：字符交集、列表交集、文件内容交集（见章末脚注）、甚至自定义类的对象——只要支持 for 和 in。

函数把"思想"（算法）从"数据"（具体变量）中解放出来

。

·初学者误区：a) 以为函数必须放在调用语句之前定义——只要调用发生时 def 已执行即可，顺序（如导入顺序）要自己保证；b) 忽略"结果可能有重复、不是数学交集"的局限——作者如实承认这个函数只是演示；c) 不知道内建 set 有 & 运算符做真正的交集——生产代码优先用内建方案。

16.14 再谈多态 (Polymorphism Revisited)

英文原文

Like all good functions in Python, intersect is polymorphic. That is, it works on arbitrary types, as long as they support the expected object interface:

```
>>> x = intersect([1, 2, 3], (1, 4)) # Pass mixed types
>>> x # Saved result object
[1]
```

 This time, we passed in different types of objects to our function—a list and a tuple (mixed types)—and it still picked out the common items. Because you don't have to specify the types of arguments ahead of time, the intersect function happily iterates through any kind of objects you send it, as long as they support the expected interfaces. For intersect, this means that the first argument has to support the for loop, and the second has to support the in membership test, in both def and lambda flavors. Any two such objects will work, regardless of their speci

fic types—that includes physically stored sequences like strings and lists; all the iterable objects we met in Chapter 14, including files and dictionaries; and even any class-based objects we code that apply operator overloading techniques we'll discuss later in the book. And here again, if we pass in objects that do not support these interfaces (e.g., numbers), Python will automatically detect the mismatch and raise an exception for us—which is exactly what we want, and the best we could do on our own if we coded explicit type tests: `>>> intersect([1, 2, 3], 1) TypeError: argument of type 'int' is not iterable` By not coding type tests and allowing Python to detect the mismatches for us, we both reduce the amount of code we need to write, and avoid artificially limiting our code's scope.

中文翻译

和 Python 中所有优秀的函数一样，`intersect` 是多态的。也就是说，只要对象支持预期的对象接口，它就能对任意类型工作：python `>>> x = intersect([1, 2, 3], (1, 4))` # 传入混合类型 `>>> x` # 保存的结果对象 `[1]` 这一次，我们向函数传入了不同类型的对象——一个列表和一个元组（混合类型）——它仍然挑出了共同的条目。因为你不必预先指定参数的类型，`intersect` 函数乐于迭代任何你送给它的对象，只要它们支持预期的接口。对 `intersect` 来说，这意味着：第一个参数必须支持 `for` 循环，第二个参数必须支持 `in` 成员测试——无论 `def` 版本还是 `lambda` 版本都是如此。任何两个这样的对象都能工作，无论它们的具体类型是什

么——包括字符串和列表这类物理存储的序列；第 14 章遇到的所有可迭代对象，包括文件（files）和字典（dictionaries）；甚至我们以后用操作符重载（operator overloading）技术编写出来的任何基于类的对象。而在这里同样，如果我们传入不支持这些接口的对象（比如数字），Python 会自动检测到不匹配并为我们抛出异常——这正是我们想要的，也是我们自己写显式类型测试能做到的最好结果：python >>> intersect([1, 2, 3], 1) TypeError: argument of type 'int' is not iterable 通过不写类型测试、让 Python 替我们检测不匹配，我们既减少了需要编写的代码量，又避免人为地限制代码的适用范围。

代码分析

```
>>> x = intersect([1, 2, 3], (1, 4))      #
>>> x                                     #
[1]
>>> intersect([1, 2, 3], 1)
TypeError: argument of type 'int' is not iterable
```

·解释器如何处理：第一次调用，seq1 被赋为列表对象 [1, 2, 3]，seq2 被赋为元组对象 (1, 4)——类型不同但接口相同。for 迭代列表取 1、2、3；1 in (1, 4) 走元组的 contains 返回 True，1 被 append；2、3 在元组中不存在，跳过。返回 [1]。第二次调用，intersect([1, 2, 3], 1)：for 正常迭代 seq1，但当执行 x in seq2 时，整数对象没有 contains 也不可迭代，解释器无法为 int 建立成员测试，抛出 TypeError: argument of type 'int' is not iterable——异常信息直接点明“int 不可迭代”。

·设计思想：错误自动检测 = 免费的防御性编程。手动写 i

`sinstance(seq2, ...)` 类型测试不仅多余，还把所有"未来可能支持 `in` 接口"的对象排除在外。让接口去工作，让类型去自由。

·易错点：混用 `def` 与 `lambda` 版本时行为一致（接口相同），所以完全可以互相替换；但注意 `lambda` 版因压缩没有可见的局部变量名（除非用嵌套 `:=` 导出），调试信息稍少。

深度理解

·核心概念：多态的两面：兼容面——任何支持接口的对象都能工作（列表/元组/字符串/字典/文件/自定义类）；失败面——不支持的自动抛异常。两者都是特性，不是 bug。

·底层实现：`for` 与 `in` 依赖的协议分别是迭代协议（`iter` 或 `getitem`）和包含协议（`contains`，缺失时退回迭代）。"支持接口"在底层 = "实现了对应特殊方法（dunder methods）"。作者在章末脚注中预告：第 30 章讲操作符重载时，自定义类可以通过 `contains`、`iter`、`getitem` 任意定义自己"迭代/成员测试"的含义。

·设计原因：为什么"不写类型测试"被提升为哲学？因为类型测试把函数冻结在"写下代码的那一刻"，而接口兼容把函数开放给"未来的整个对象宇宙"。Python 社区由此衍生出"鸭子类型"与"协议（protocol）"思想。

·解决什么实际问题：一个 `intersect` 通吃字符串、列表、元组、字典键、文件行（注意脚注：文件是单遍迭代器，读到文件尾后需要 `file.seek(0)` 或重新 `open` 才能再遍历

)。框架代码尤其受益：一次编写，全部类型共享。

·初学者误区：a) 想当然认为 `intersect([1,2,3], 1)` 应该返回空列表——不，它直接报错，这才是预期行为（接口不兼容就该大声失败）；b) 以为文件对象可以随便多次 `in/` 迭代——单遍迭代器是新手常踩的坑；c) 看到异常就慌——Python 哲学是“let it crash”式地尽早暴露错误，配合异常处理（第 33-35 章）使用。

16.15 过渡：局部变量 (Segue: Local Variables)

英文原文

Perhaps the most interesting part of this example, though, is its names. It turns out that the variable `res` inside the `def` version of `intersect` is what in Python is called a local variable—a name that is visible only to code inside the function `def` and that exists only while the function call runs. In fact, because all names assigned in any way inside a function are classified as local variables by default, nearly all the names in the `def` qualify:— `res` is obviously assigned, so it is a local variable.— Arguments are passed by assignment, so `seq1` and `seq2` are, too.— The `for` loop assigns items to a variable, so the name `x` is also local. All these local variables appear when the function is called and disappear when the function exits—the `return` statement at the

end of intersect sends back the result object, but the name res goes away. The same goes for argument names in the lambda, though its compression hides its loop variable (sans a nested := to export it). Because of this, a function's variables don't clash with names elsewhere, but also won't remember values between calls. Although the object returned by a function lives on, retaining other state information requires other techniques. To fully explore the notion of locals and state, though, we need to move on to the scopes coverage of the next chapter.

中文翻译

不过，这个示例最有趣的部分也许是它的名字。原来，def 版 intersect 里的变量 res 在 Python 中被称为局部变量（local variable）——一个只能被函数 def 内部的代码看到、并且只在函数调用运行期间存在的名字。事实上，因为在函数内部以任何方式赋值的所有名字默认都被归类为局部变量，所以 def 里几乎所有的名字都符合条件：- res 显然是赋值的，所以它是局部变量。- 参数通过赋值传递，所以 seq1 和 seq2 也是局部变量。- for 循环把条目赋给一个变量，所以名字 x 也是局部变量。所有这些局部变量在函数被调用时出现，在函数退出时消失——intersect 末尾的 return 语句把结果对象送回，但名字 res 不见了。lambda 版本中的参数名也一样，只不过它的压缩形式隐藏了循环变量（除非用嵌套的 := 把它导出）。正因为如此，函数的变量不会和别处的名字冲突，但也不会两次调用之间记住值。虽然函数返回的对象继续活着，但要保留其他状态信息（state i

nformation) 就需要其他技巧了。不过，要完全探索"局部变量与状态"的概念，我们需要进入下一章的作用域 (scopes) 内容。

代码分析

```
def intersect(seq1, seq2):  
    res = [] #  
    for x in seq1: # seq1  
        if x in seq2: #  
            res.append(x) #  
    return res # res
```

·解释器如何处理：调用开始时，解释器为这次调用分配一个帧 (frame)，帧内维护局部命名空间字典。三个名字进入其中：seq1、seq2 (实参赋值产生)、res (函数体第一条赋值产生)；循环变量 x 每次迭代被重新赋值 (同一个名字，新的对象)。return 执行时，列表对象被引用计数保住性命并交还调用方；帧被弹出销毁，局部字典随之释放——于是"名字 res 消失了，对象还活着"。注意：lambda 版的压缩写法没有可见的 res/x 名字，但底层同样有临时变量，只是被表达式求值隐藏了 (若用海象运算符 := 可以显式导出循环变量)。

·设计思想：局部默认 (local-by-default) 是作用域规则的第一定律：函数里写的名字，函数退出就没了。这既是"隔离" (不污染外界) 也是"遗忘" (不留状态)。

·易错点：如果函数内想"记住"上一次调用的值，直接用局部变量不行 (每次调用都重新初始化) ——需要闭包 (nonlocal)、全局变量、或可变对象默认参数等技巧，第 17 章详述。

深度理解

·核心概念：局部变量 = "仅在调用期间存在、仅函数内可见"的名字。判定规则简单粗暴：函数内凡是被赋值的名字都是局部变量——包括参数、循环变量、显式赋值。这是 Python 作用域规则的"默认值"，第 17 章的整章铺垫。

·底层实现：每次函数调用创建新帧（frame，在调用栈上），帧持有局部命名空间。CPython 3.x 用"快速局部变量数组"（fast locals）优化：编译期就把局部变量定死为帧内固定槽位，运行时按槽位索引存取，比字典查找快得多——这也是 Python 循环内访问局部变量最快的经典原因。

·设计原因：为什么默认全部局部？这是安全与简单的权衡：如果函数内随意写名字会改到全局，那么不小心写错一个变量名就会悄悄污染整个程序——像在公共图书馆里乱涂乱画。默认局部让"误伤"几乎不可能，想要全局必须显式声明（global），这是"显式优于隐式"（Explicit is better than implicit）哲学的体现。

·解决什么实际问题：函数之间变量隔离 = 模块化开发的根基：不同的函数可以用相同的变量名（i、x、res、temp）而不互相踩踏；这也让函数成为可独立测试的单元。

·初学者误区：a) 在函数里读一个不存在的全局名不会报错（会沿着 LEGB 向外查找，最终 NameError），但在函数里给全局名赋值不会改全局，而是静默创建一个同名的局部变量——第 17 章经典大坑；b) 以为"函数返回后变量还在"——不，只有对象活着，名字没了；c) 想当然认

为循环结束后 `x` 还保留最后的值——在函数内部调用期间确实保留，但函数一退出就没了。

16.16 本章小结 (Chapter Summary)

英文原文

This chapter introduced the core ideas behind function definition—the syntax and operation of the `def` and `return` statements and lambda expression, the behavior of function call expressions, and the notion and benefits of polymorphism in Python functions. As we saw, a `def` and lambda are executable code that create a function object at runtime; when the function is later called, objects are passed into it by assignment (which means object reference in Python), and computed values are sent back by `return` in `def` and implicitly in lambda. We also began exploring the concepts of local variables and scopes, but saved the details for Chapter 17. First, though, a quick quiz.

中文翻译

本章介绍了函数定义背后的核心思想——`def` 和 `return` 语句与 lambda 表达式的语法和操作、函数调用表达式的行为，以及 Python 函数中多态的概念与好处。正如我们所看到的，`def` 和 lambda 是创建函数对象的可执行代码，在运行时生效；当函数之后被调用时，对象通过赋值传入（在 Python 中意味着按对象引用传递），计算出的值由 `def` 中

的 return 送回、在 lambda 中则是隐式送回。我们还开始探索了局部变量和作用域的概念，但把细节留给了第 17 章。不过，先来个小小测验。

深度理解

·核心概念：本章链条可压缩为一句口诀：def 运行时造对象 → 调用按引用传参 → return 送回结果 → 类型全靠多态 → 函数内名字皆局部。这五句话是全部函数话题的骨架。

·底层实现：把五句话映射到 CPython 机制：MAKEFUNCTION (def 指令) → 实参引用赋值 (帧内局部槽位) → RETURNVALUE → 动态方法分派 (BINARYOP 等) → 帧生命周期管理。看到这里，你已经具备“用实现眼光看函数”的初步能力。

·设计原因：作者刻意把“局部变量/作用域”只开个头就停住，是为了先建立“函数会隔离名字”的直觉，再把全套规则 (LEGB、global/nonlocal) 交给第 17 章——教学上的“先给结论、后给原理”。

·解决什么实际问题：本章之后，你可以开始用函数重构自己的脚本：把重复循环提取为工具函数，利用多态让它通吃多种数据类型，并安心使用局部变量而不必担心命名冲突。

·初学者误区：急于把所有高级工具 (nonlocal、yield、async) 都弄明白——本章只需牢固掌握 def/return/lambda/多态/局部变量五件事，其余留给后续章节自然消化。

16.17 测验 (Test Your Knowledge: Quiz)

英文原文

1. What is the point of coding functions? 2. At what time does Python create a function? 3. What does a def function return if it has no return statement in it? 4. When does the code nested inside the function definition run? 5. What's wrong with checking the types of objects passed into a function?

中文翻译

1. 编写函数的意义是什么？ 2. Python 在什么时刻创建函数？ 3. 如果一个 def 函数内部没有 return 语句，它返回什么？ 4. 函数定义内部嵌套的代码在什么时候运行？ 5. 检查传入函数的对象的类型有什么问题？

16.18 测验答案 (Test Your Knowledge: Answers)

英文原文

1. Functions are the most basic way of avoiding code redundancy in Python—factoring code into functions means that we have only one copy of an operation's code to update in the future. Functions are also the basic unit of code reuse in Python—wrapping code in functions makes it a reusable tool, callable in a variety

y of programs. Finally, functions allow us to divide a complex system into manageable parts, each of which may be developed individually. Functions are also used for object methods, but we're postponing this role until we study classes.

2. A function is created when Python reaches and runs a `def` statement, which creates a function object and assigns it to the function's name. This normally happens when the enclosing module file is imported by another module (recall that imports run the code in a file from top to bottom, including any `defs`), but it can also occur when a `def` is typed interactively or nested in other statements, such as `ifs`. Functions are also created when a `lambda` is reached and run, though this doesn't normally happen during an import operation.

3. A `def` function returns the `None` object by default if the control flow falls off the end of the function body without running into a `return` statement. Such functions are usually called with expression statements, as assigning their `None` results to variables is generally pointless. A `return` statement with no expression in it also returns `None`. A `lambda` implicitly returns its expression's result, so it has no default.

4. The function body (the code nested inside the function `def` statement or following the colon in `lambda`) is run when the function is later invoked with a call expression. The body runs anew each time the function is called.

5. Checking the types of objects passed into a function effectively breaks the fun

ction's flexibility, constraining the function to work on specific types only. Without such checks, the function would likely be able to process an entire array of object types—any objects that support the interface expected by the function will work. (The term interface means the set of methods and expression operators the function's code runs.)

中文翻译

1. 函数是 Python 中避免代码冗余 (redundancy) 的最基本方式——把代码提炼成函数意味着将来要修改一个操作时，我们只有一份代码需要更新。函数也是 Python 中代码复用 (reuse) 的基本单元——把代码包进函数，它就变成可复用的工具，可以在各种程序中调用。最后，函数让我们把复杂系统分解成可管理的部分，每个部分都可以单独开发。函数还用于对象方法 (methods)，但这个角色我们要等学习类时再展开。2. 当 Python 执行到并运行一条 def 语句时，函数被创建——该语句创建一个函数对象并把它赋给函数名。这通常发生在包含它的模块文件被另一个模块导入时（回想一下：导入会从上到下运行文件里的代码，包括其中的所有 def），但也可能发生在 def 被交互式输入、或嵌套在其他语句（比如 if）中时。当 lambda 被执行到时，函数也会被创建——尽管这通常不会发生在导入操作期间。3. 如果控制流走出函数体末尾而没有碰到 return 语句，def 函数默认返回 None 对象。这类函数通常用表达式语句调用，因为把它们的结果赋给变量通常毫无意义。不带表达式的 return 语句同样返回 None。而 lambda 会隐式返回其表达式的结果，所以它没有“默认值”一说。

。4. 函数体（嵌套在函数 `def` 语句内部、或位于 `lambda` 冒号之后的代码）在函数之后被调用表达式（call expression）调用时运行。每次调用函数，函数体都会重新运行一遍。5. 检查传入函数的对象的类型，实际上会破坏函数的灵活性，把函数限制为只能处理特定类型。没有这些检查，函数通常能够处理一整类对象类型——任何支持函数预期接口的对象都能工作。（术语接口（interface）指函数代码要运行的方法和表达式运算符的集合。）

深度理解

·核心概念：五道题正好覆盖本章五根支柱：函数意义（复用/分解）、创建时机（`def` 运行时执行）、默认返回值（`None`）、执行时机（调用时才运行函数体）、类型检查之害（破坏灵活性）。答对五题，本章就及格了。

·底层实现：第 2 题与第 4 题是一体两面：`def` 创建函数对象是“立即”的（执行到该行），函数体执行是“延迟”的（调用时）。导入模块会顺带执行顶层 `def`——所以模块导入后函数就可用。第 3 题：字节码层面没有 `return` 的函数在帧结束时由解释器自动压入 `None`。

·设计原因：第 5 题的答案揭示 Python 的核心价值主张：用测试换灵活（test-driven flexibility）。Java/C++ 用编译期类型检查换安全，Python 用运行时多态换简洁——两者各有所长，Python 选择信任接口而非类型。

·解决什么实际问题：这些知识直接指导日常写码：写工具函数时不要加类型守卫；忘写 `return` 时要知道函数悄悄返回 `None` 而可能是 bug 根源；把函数放进模块以便导

入复用。

·初学者误区：第 3 题最易答错——很多人以为"没有 return 就什么都不返回"，实际返回 None。第 5 题最反直觉——"检查类型难道不是更严谨吗？"作者的回答：那是以牺牲通用性为代价的虚假严谨。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

- 通用工具函数：把重复逻辑（数据清洗、字符串处理、校验）提取为模块级函数，全项目共享——正是本章 `intersect` 的工程原型。
- 多态驱动的框架设计：Web 框架 (Flask/Django) 的视图函数、ORM 的模型方法、日志系统的 handler——框架只定义"接口（协议）"，具体对象自行实现，这就是本章多态思想的大型化应用。
- 回调与延迟执行：GUI/异步框架里把函数当作参数传递（`onclick=handleclick`）；`sorted(data, key=len)` 把函数对象作为 key——"函数即值"（16.9 节）的直接商业价值。
- 函数属性做缓存/元数据：`func.attr = value` 的实战形态是装饰器里给函数挂 `wrapped`、缓存计数等元数据（第 6 部分展开）。

与其他语言 (Java/C++) 的区别

维度	Python	C	Java
函数定义时机	运行时执行 def , 动态创建函数对象	编译期静态编译 , 运行前必须完整定义	编译期 (字节码) , 类加载时绑定
函数是否为对象	一等对象, 可传参、可存容器、可挂属性	函数指针勉强模拟, 不能挂属性	需包在接口/函数式接口里 (Java 8 起)
类型约束	零声明, 运行时多态分派	强类型, 编译期检查	强类型, 编译期检查 + 泛型
传参语义	按对象引用传递 (传引用, 不复制)	按值传递 (可传指针模拟引用)	基本类型按值、对象按引用值
默认返回值	无 return 返回 None	无返回类型则 void (无值)	无返回类型则 void
重载	无重载 (同名即覆盖), 靠多态	支持函数重载	支持方法重载

核心差异一句话: C/Java 在"写代码时"就定死类型, Python 在"运行到那行时"才问对象"你会做这个吗? "

Python 发展历史背景

- lambda 是 1991 年 Python 1.0 发布时的原始特性之一 (受 Lisp/函数式语言启发) , 而海象运算符 := 到 2020 年 Python 3.8 才加入——作者的"四个表达式等价物是随时间累积的"正是这段历史。
- 函数属性的支持 (func.attr = value) 源自 Python 早期 "一切皆对象"的坚持, 比 Java 的反射包袱轻得多。
- 动态类型 + 多态一直是 Python 的立身之本; 2008 年后社区流行的类型注解 (PEP 484, 2014 年) 以"不影响运行时的文档"形式回归, 佐证了本书"注解不被解释器使用"的说法。

高级开发者需要掌握的相关知识

- 闭包与延迟绑定：lambda 捕获循环变量、函数工厂返回值捕获外层变量——第 17 章作用域必修。
- 参数传递全谱：位置/关键字/默认值/args/kwargs 的完整匹配规则——第 18 章。
- 函数式工具：map/filter/reduce、生成器表达式、functools.partial——第 19 章。
- 生成器与协程：yield/await 让"函数"升级为"挂起-恢复的流水线"——第 20 章。
- 字节码视角：dis.dis(func) 查看 MAKEFUNCTION/RETURNVALUE/BINARYOP 指令，能让你对"函数对象"有显微镜级的理解。
- 性能常识：局部变量访问最快（fast locals），函数调用有帧开销——高并发热点代码里，把调用移出内循环是基本优化。

学习建议 (Learning Advice)

- 重要程度：（5/5 星）——函数是 Python 一切结构的母体，类、装饰器、生成器全部建立在它之上；本章的"def 运行时执行"与"按引用传参"两个概念理解不牢，后续会持续踩坑。
- 应该掌握到什么程度：
 1. 背诵级：def 是运行时语句，执行到才创建函数对象；函数是一等对象，可改名、存容器、挂属性。

2. 熟练级：手写 `def/return/lambda` 并在 REPL 验证；能解释 `times('Py', 4)` 为何是 `'PyPyPyPy'`；能讲清“传对象引用但不产生名字别名”。

3. 应用级：能把自己的脚本改写成含工具函数的模块；能指出函数内哪些名字是局部变量、生命周期如何。

4. 思辨级：能说明为什么“检查参数类型”是反模式，以及何时（特殊需求）例外。

·后续应该学习哪些相关内容：

·立即：第 17 章作用域（LEGB、`global/nonlocal`、闭包）——理解“局部变量”的完整规则。

·紧接着：第 18 章参数传递（按位置/关键字、默认值、`args/kwargs`、解包）。

·然后：第 19 章函数式工具与 `lambda` 的实战、第 20 章生成器（`yield`）。

·远期：第六部分类——“类是函数包的组合”，届时函数最终演化为方法。

本章一句话总纲：函数是运行时创建的对象，按引用传参、按接口协作、按局部隔离——理解了这四个“按”，你就掌握了 Python 函数的灵魂。